Programación III - Ejercicios de Práctica

Parte 1: Seguimiento de Código y Conceptos

Ejercicio 1

Dado el siguiente código basado en `fileSync.js`:

```
const fs = require('fs');
console.log('1. Inicio lectura');
const data = fs.readFileSync('myfile.txt', 'utf8');
console.log('2. Contenido:', data);
console.log('3. Fin lectura');
```

Indique el orden de salida por consola considerando que el archivo contiene `"HOLA PROGRA 3"`. Explique brevemente si la ejecución es bloqueante o no bloqueante.

Ejercicio 2

Dado el siguiente código basado en `fileAsync.js`:

```
const fs = require('fs');
console.log('1. Inicio lectura');
fs.readFile('myfile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('2. Contenido:', data);
});
console.log('3. Fin lectura');
```

Consigna: Indique el orden exacto de impresión en consola y justifique según el modelo de I/O de Node.js.

Ejercicio 3

Analice el siguiente bloque de código:

```
const original = [10, 2, 3];
original.map(aux => {
  aux *= 2;
  console.log(aux);
  return aux;
});
console.log(original);
```

Consigna:

1. ¿Qué valores se imprimen en consola?

2. ¿Sufre modificaciones el arreglo `original`? Justifique.

Ejercicio 4

En relación al siguiente fragmento:

```
async function obtenerDniAsync() {
  return "12345678";
}
```

Consigna: ¿Qué retorna exactamente la llamada a `obtenerDniAsync()`? ¿Cómo debe consumirse dicho valor desde la función llamadora?

Ejercicio 5

Dado el código:

```
fetch("https://ejemplo.local/api/datos2.txt")
  .then(res => res.text())
  .then(data => console.log(data))
  .catch(err => console.log("Error de conexión"));

console.log("Estoy despues de fetch");
```

Consigna: Indique el orden de salida por consola al ejecutar el script.

Ej1 Respuesta:

- Orden de salida:
  1. `1. Inicio lectura`
  2. `2. Contenido: HOLA PROGRA 3`
  3. `3. Fin lectura`
- Explicación: La ejecución es **bloqueante (síncrona)**. `fs.readFileSync` detiene el hilo principal de ejecución hasta completar la lectura del archivo en disco antes de pasar a la línea siguiente.

Ej2 Respuesta:

- Orden de salida:
  1. `1. Inicio lectura`
  2. `3. Fin lectura`
  3. `2. Contenido: HOLA PROGRA 3`

- Justificación: La ejecución es **no bloqueante (asíncrona)**. `fs.readFile` delega la operación de I/O al sistema y continúa ejecutando el hilo principal. El callback registrado se ejecuta en el Event Loop una vez finalizada la lectura del archivo.

Ej3 Respuesta:

1. Se imprimen consecutivamente: `20`, `4`, `6` y finalmente el arreglo `[ 10, 2, 3 ]`.
2. No, el arreglo `original` no sufre modificaciones. El método `.map()` crea y retorna un nuevo arreglo sin alterar el arreglo sobre el cual se invoca (inmutabilidad).

Ej 4 Respuesta:

- Retorna una **Promesa en estado cumplido (fulfilled)** cuyo valor resuelto es `"12345678"`.
- Para consumir dicho valor se debe utilizar la sintaxis `await obtenerDniAsync()` dentro de una función asíncrona, o encadenar un `.then(dni => ...)` desde el contexto llamador.

Ej 5 Respuesta:

- Se imprime primero `"Estoy despues de fetch"`.
- **Posteriormente, una vez resuelta o rechazada la Promesa de la petición HTTP, se imprime el contenido recibido o `"Error de conexión"`.**

Parte 2: Desarrollo e Implementación

Ejercicio 6: Procesamiento de archivos con Promesas (`fs/promises`)

Consigna:

Escribir la función `procesarPrecios(rutaArchivo)` utilizando el módulo `fs/promises`.
Requerimientos:

1. Leer un archivo de texto con un precio numérico por línea.
2. Mediante encadenamiento de `.then()`:
    a. Convertir el texto en un array de números.
    b. Filtrar los precios mayores a 100.
    c. Aplicar un 10% de descuento a cada precio resultante.
3. Imprimir el array final.

4. Capturar errores con `.catch()`.

Solución de referencia:

```
const fs = require('fs/promises');

function procesarPrecios(rutaArchivo) {
  fs.readFile(rutaArchivo, 'utf-8')
    .then(contenido => contenido.split('\n'))
    .then(lineas => lineas.map(linea => Number(linea.trim())))
    .then(precios => precios.filter(precio => precio > 100))
    .then(preciosFiltrados => preciosFiltrados.map(precio => precio * 0.9))
    .then(preciosConDescuento => {
      console.log('Precios finales con descuento:', preciosConDescuento);
    })
    .catch(err => {
      console.error('Error al procesar el archivo de precios:', err.message);
    });
}
```

Ejercicio 7: Petición HTTP con `async / await`

Consigna:

Escribir la función asíncrona `obtenerAlumnosDestacados(url)` que cumpla con:

1. Realizar la petición `fetch()` dentro de un bloque `try...catch`.Realizar la petición fetch() dentro de un bloque try...catch.
2. Verificar la propiedad `ok` de la respuesta (lanzar error en caso contrario).
3. Convertir la respuesta con `.json()`.
4. Retornar e imprimir un array con los nombres (`nombre`) de aquellos alumnos con nota (`nota`) mayor o igual a 7.

Solución de referencia:

```
async function obtenerAlumnosDestacados(url) {
  try {
    const respuesta = await fetch(url);

    if (!respuesta.ok) {
      throw new Error(`Error en el servidor: HTTP ${respuesta.status}`);
    }

    const alumnos = await respuesta.json();
```

```
    const destacados = alumnos
      .filter(alumno => alumno.nota >= 7)
      .map(alumno => alumno.nombre);

    console.log("Alumnos destacados:", destacados);
    return destacados;

  } catch (error) {
    console.error("Ocurrió un error al consultar la API:", error.message);
  }
}
```

Ejercicio 8: Manejo de Callbacks (`fs` tradicional)

Consigna:

Escribir la función `contarLineas(rutaArchivo)` utilizando `fs.readFile` con la firma estándar *Error-First Callback*.

Requerimientos:

1. Manejar el parámetro de error imprimiendo un mensaje y finalizando la función en caso de falla.
2. Si la lectura es exitosa, contar las líneas del archivo e imprimir: `"El archivo [rutaArchivo] tiene X líneas."`.

Solución de referencia:

```
const fs = require('fs');

function contarLineas(rutaArchivo) {
  fs.readFile(rutaArchivo, 'utf-8', (err, data) => {
    if (err) {
      console.error('Error al leer el archivo:', err.message);
      return;
    }

    const lineas = data.split('\n');
    console.log(`El archivo ${rutaArchivo} tiene ${lineas.length} líneas.`);
  });
}
```

Ejercicio 9: Refactorización a `async / await`

Consigna:

Refactorizar la siguiente función basada en `.then()` a la sintaxis `async / await` con bloque `try...catch`:

```
function cargarDatos() {
  fetch("https://ejemplo.local/api/datos2.txt")
    .then(res => {
      if (!res.ok) throw new Error("Error HTTP " + res.status);
      return res.text();
    })
    .then(texto => {
      console.log("Contenido recibido:", texto);
    })
    .catch(err => {
      console.error("Error:", err.message);
    });
}
```

Solución de referencia:

```
async function cargarDatos() {
  try {
    const res = await fetch("https://ejemplo.local/api/datos2.txt");

    if (!res.ok) {
      throw new Error("Error HTTP " + res.status);
    }

    const texto = await res.text();
    console.log("Contenido recibido:", texto);

  } catch (err) {
    console.error("Error:", err.message);
  }
}
```